Despliegue del SOC con Wazuh

Grupos, configuración centralizada, agentes y usuarios del dashboard
Práctica 1 (UT3) - IC 5021 - Empresa de referencia: TECNORIBERA, S.L.

Fecha de realización	22/10/2026
Equipo (host SOC)	paolo
Plataforma	Wazuh 4.14.8 (manager, indexer y dashboard) en contenedores Docker
Carpeta de trabajo	~/wazuh-soc/plataforma

1. Qué se pedía y qué recoge este informe

La práctica consiste en montar el SOC (centro de vigilancia de seguridad) de una empresa de 18 personas: política de nombres, grupos por tipo de equipo, configuración distinta según el riesgo, agentes dados de alta, usuarios por rol y, después, alertas y una regla propia.

Este informe recoge **lo realizado hasta ahora**: preparación del equipo, creación de los cuatro grupos, los cuatro ficheros `agent.conf`, el alta de dos agentes y los usuarios del dashboard, junto con los **problemas que surgieron y cómo se resolvieron**. Por indicación del profesor, los agentes se han creado solo con su nombre y su grupo; las máquinas o contenedores donde se instalarán llegarán más adelante. Por eso las alertas y la regla local quedan pendientes (sección 6).

2. Entorno de trabajo

Sistema	Ubuntu Linux (kernel 7.0.0-38), arquitectura x86_64
Memoria RAM	14 GB en total, 8,7 GB disponibles (la guía pide 8 GB libres)
Disco	42 GB libres (la guía pide 20 GB)
vm.max_map_count	1048576 (supera el mínimo de 262144, no hubo que cambiarlo)
Docker	Docker 29.1.3 y Compose v5.5.1 (se usa el comando `docker-compose`)
Contenedores	plataforma-wazuh.manager-1, plataforma-wazuh.indexer-1, plataforma-wazuh.dashboard-1
Red de Docker	plataforma_default
Acceso al panel	https://localhost (certificado autofirmado del laboratorio)

3. Pasos realizados

3.1 Comprobar los requisitos del equipo

Por qué: si falta memoria, espacio o el ajuste del kernel, el indexer (el almacén de alertas) puede no arrancar y los errores resultan confusos.

```
uname -a
docker --version
free -h
df -h /
sysctl vm.max_map_count
```

Resultado: todos los requisitos se cumplen (ver tabla de la sección 2).

3.2 Comprobar que el stack de Wazuh está arriba

Por qué: los grupos y los agentes se crean contra el manager, así que debe estar en marcha.

```
cd ~/wazuh-soc/plataforma
sudo docker-compose ps
```

Resultado: los tres contenedores (manager, indexer y dashboard) aparecen como *Up*.

3.3 Crear los cuatro grupos

Por qué: un grupo es una "carpeta de configuración" compartida por varios agentes. Se agrupa por **tipo de equipo** y no por persona, porque las personas cambian de departamento y los roles casi no cambian. Se crean **antes** que los agentes para que cada agente nazca ya en su grupo.

```
sudo docker exec -it plataforma-wazuh.manager-1 bash
/var/ossec/bin/agent_groups -a -g ofimatica -q
/var/ossec/bin/agent_groups -a -g direccion -q
/var/ossec/bin/agent_groups -a -g desarrollo -q
/var/ossec/bin/agent_groups -a -g servidores -q
/var/ossec/bin/agent_groups -l
```



Figura 1. Lista de grupos: 'default' y los cuatro nuevos. Los nuevos aparecen con 0 agentes.

3.4 Configuración centralizada por grupo (agent.conf)

Idea: en lugar de tocar cada equipo, se publica desde el manager un fichero `agent.conf` por grupo y todos los agentes del grupo lo reciben solos. Así se vigila de forma distinta según el riesgo: **a mayor impacto de un ataque, más vigilancia**. El contenedor del manager no tiene *nano*, así que los ficheros se escribieron con `cat > fichero << 'EOF'` y se comprobaron después con `cat`.

```
cat > /var/ossec/etc/shared/ofimatica/agent.conf << 'EOF'
...contenido del fichero...
EOF
cat /var/ossec/etc/shared/ofimatica/agent.conf
```

Resumen de lo que vigila cada grupo (todos llevan además la etiqueta `location = valencia-sede`):

Grupo	Equipos	Criticidad	Revisión completa	Qué vigila	Guarda cambios
ofimatica	Windows de puesto (FIN, RRH, OPE, COM)	media	cada 12 h	C:\Users y la carpeta StartUp. Ignora C:\Users\All Users	No
direccion	Windows de dirección	muy_alta	cada 6 h	C:\Users, StartUp y C:\Program Files	Sí
desarrollo	Linux de los desarrolladores	alta	cada 6 h	/home, /etc y /opt/ic-lab/monitor. Ignora .cache y node_modules	Solo en /etc
servidores	Linux de servicio (ERP y fileserver)	muy_alta	cada 4 h	/etc, /root, /var/www y /opt. Ignora /var/log. Lee /var/log/auth.log	Sí

Por qué estas diferencias:

- **Frecuencia:** dirección y servidores se revisan más a menudo porque son el objetivo más valioso. En ofimatica más revisiones solo serían ruido.
- **report_changes:** guarda *qué* cambió dentro del fichero, no solo que cambió. Es muy útil pero genera mucha más información, por eso solo se activa donde compensa.
- **StartUp** (Windows) es una vía clásica para que el malware se ejecute al iniciar sesión. **/var/www**, **/root** y **/opt** (servidores) son sitios donde no debería aparecer nada sin que lo sepa IT.
- **Ignorar carpetas** (.cache, node_modules, /var/log) evita llenar el SOC de alertas inútiles: una buena política también decide qué no vigilar.
- **auth.log** (servidores) es el fichero donde Linux anota los accesos SSH, el uso de sudo y los cambios de usuarios. Es la fuente de identidad del servidor.

```
bash-5.2# cat /var/ossec/etc/shared/ofimatica/agent.conf
<agent_config>
  <labels>
    <label key="criticality">media</label>
    <label key="location">valencia-sede</label>
  </labels>
  <syscheck>
    <frequency>43200</frequency>
    <directories check_all="yes" realtime="yes">C:\Users</directories>
    <directories check_all="yes" realtime="yes">C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp</d
irectories>
    <ignore>C:\Users\All Users</ignore>
  </syscheck>
</agent_config>
bash-5.2#
```

Figura 2. agent.conf del grupo ofimatica.

```
bash-5.2# cat /var/ossec/etc/shared/direccion/agent.conf
<agent_config>
  <labels>
    <label key="criticality">muy_alta</label>
    <label key="location">valencia-sede</label>
  </labels>
  <syscheck>
    <frequency>21600</frequency>
    <directories check_all="yes" realtime="yes" report_changes="yes">C:\Users</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">C:\ProgramData\Microsoft\Windows\Start Men
u\Programs\StartUp</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">C:\Program Files</directories>
  </syscheck>
</agent_config>
```

Figura 3. agent.conf del grupo direccion (con report_changes y Program Files).

```
bash-5.2# cat /var/ossec/etc/shared/desarrollo/agent.conf
<agent_config>
  <labels>
    <label key="criticality">alta</label>
    <label key="location">valencia-sede</label>
  </labels>
  <syscheck>
    <frequency>21600</frequency>
    <directories check_all="yes" realtime="yes">/home</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">/etc</directories>
    <directories check_all="yes" realtime="yes">/opt/ic-lab/monitor</directories>
    <ignore>/home/*/.cache</ignore>
    <ignore>/home/*/node_modules</ignore>
  </syscheck>
</agent_config>
```

Figura 4. agent.conf del grupo desarrollo (con carpetas ignoradas).

```
bash-5.2# cat /var/ossec/etc/shared/servidores/agent.conf
<agent_config>
  <labels>
    <label key="criticality">muy_alta</label>
    <label key="location">valencia-sede</label>
  </labels>
  <syscheck>
    <frequency>14400</frequency>
    <directories check_all="yes" realtime="yes" report_changes="yes">/etc</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">/root</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">/var/www</directories>
    <directories check_all="yes" realtime="yes" report_changes="yes">/opt</directories>
    <ignore>/var/log</ignore>
  </syscheck>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/auth.log</location>
  </localfile>
</agent_config>
```

Figura 5. agent.conf del grupo servidores (con lectura de auth.log).

3.5 Validar los ficheros

Por qué: estos ficheros son XML y una etiqueta mal cerrada hace que el manager los rechace. Es mejor descubrirlo ahora que cuando haya agentes esperando su configuración.

```
sudo docker exec plataforma-wazuh.manager-1 /var/ossec/bin/verify-agent-conf
```

```
paolo@paolo:~/wazuh-soc/plataforma$ sudo docker exec plataforma-wazuh.manager-1 /var/ossec/bin/verify-agent-con
f

verify-agent-conf: Verifying [etc/shared/default/agent.conf]
verify-agent-conf: OK

verify-agent-conf: Verifying [etc/shared/desarrollo/agent.conf]
verify-agent-conf: OK

verify-agent-conf: Verifying [etc/shared/direccion/agent.conf]
verify-agent-conf: OK

verify-agent-conf: Verifying [etc/shared/ofimatica/agent.conf]
verify-agent-conf: OK

verify-agent-conf: Verifying [etc/shared/servidores/agent.conf]
verify-agent-conf: OK
```

Figura 6. Validación: los cinco ficheros (default y los cuatro grupos) salen como OK.

3.6 Dar de alta los agentes (nombre y grupo)

Qué significa: dar de alta un agente es crear su "ficha" en el manager (ID, nombre y grupo), sin que exista todavía ningún programa funcionando detrás. Es como dar de alta a un empleado antes de que venga a trabajar. Por eso aparecen como *Never connected* (nunca conectado). Los nombres siguen la política DEPARTAMENTO-FUNCIÓN-NN, para que a las 3 de la mañana un analista sepa de qué equipo se trata solo con leer el nombre.

Se crearon con la herramienta `manage_agents` del manager (nombre del agente e IP *any*) y después se asignó cada uno a su grupo:

```
/var/ossec/bin/agent_control -l
/var/ossec/bin/agent_groups -a -i 002 -g servidores -q
/var/ossec/bin/agent_groups -a -i 003 -g desarrollo -q
/var/ossec/bin/agent_groups -s -i 002
/var/ossec/bin/agent_groups -s -i 003
/var/ossec/bin/agent_groups -l
```

```
Wazuh agent_control. List of available agents:
   ID: 000, Name: wazuh.manager (server), IP: 127.0.0.1, Active/Local
   ID: 001, Name: paolo, IP: any, Active
   ID: 002, Name: IT-SRV-ERP, IP: any, Never connected
   ID: 003, Name: DEV-NB-01, IP: any, Never connected
```

Figura 7. Lista de agentes: IT-SRV-ERP (002) y DEV-NB-01 (003) sin conectar. El 001 'paolo' es un agente de prueba anterior.

```
bash-5.2# /var/ossec/bin/agent_groups -a -i 002 -g servidores -q
/var/ossec/bin/agent_groups -a -i 003 -g desarrollo -q
Group 'servidores' added to agent '002'.
Group 'desarrollo' added to agent '003'.
```

Figura 8. Asignación de cada agente a su grupo.

```
bash-5.2# /var/ossec/bin/agent_groups -s -i 002
/var/ossec/bin/agent_groups -s -i 003
/var/ossec/bin/agent_groups -l
The agent 'IT-SRV-ERP' with ID '002' belongs to groups: servidores.
The agent 'DEV-NB-01' with ID '003' belongs to groups: desarrollo.
Groups (5):
  default (1)
  desarrollo (1)
  direccion (0)
  ofimatica (0)
  servidores (1)
Unassigned agents: 0.
```

Figura 9. Comprobación: cada agente pertenece solo a su grupo y los contadores cuadran.

ID	Nombre	Grupo	Estado
002	IT-SRV-ERP (servidor Linux del ERP)	servidores	Never connected
003	DEV-NB-01 (portátil Linux de desarrollo)	desarrollo	Never connected

El agente de Windows (grupo ofimatica) no se ha creado por ahora, por indicación del profesor.

3.7 Usuarios del dashboard por rol

Por qué: un SOC real no comparte el usuario *admin*, por dos motivos: **seguridad** (su contraseña por defecto es pública) y **trazabilidad** (si todos usan admin, no se sabe quién hizo qué). Se crearon desde Indexer management > Security > Internal users > Create internal user.

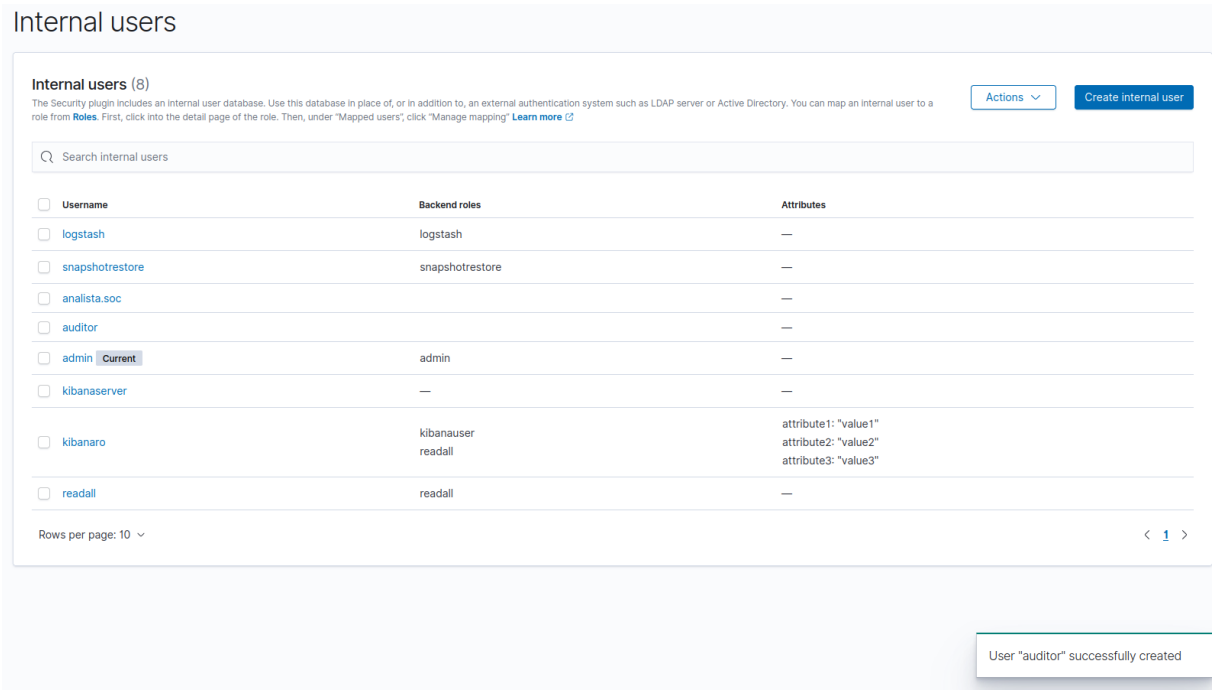


Figura 10. Lista de usuarios internos con analista.soc y auditor recién creados.

Los permisos vienen de los **roles**, no del usuario. Se eligieron roles que ya trae el sistema y se evitaron a propósito los que dan control de administración (all_access y los *_full_access):

Usuario	Roles	Cómo se asignaron
analista.soc	readall_and_monitor y kibana_user (lee todo, ve el estado del clúster y puede usar el panel)	Por nombre de usuario (pestaña Mapped users del rol)
auditor	readall (lectura) y kibana_read_only (solo lectura en la interfaz). También se mapeó en readall-copy, una copia del rol readall	readall mediante el backend role del usuario; kibana_read_only por nombre

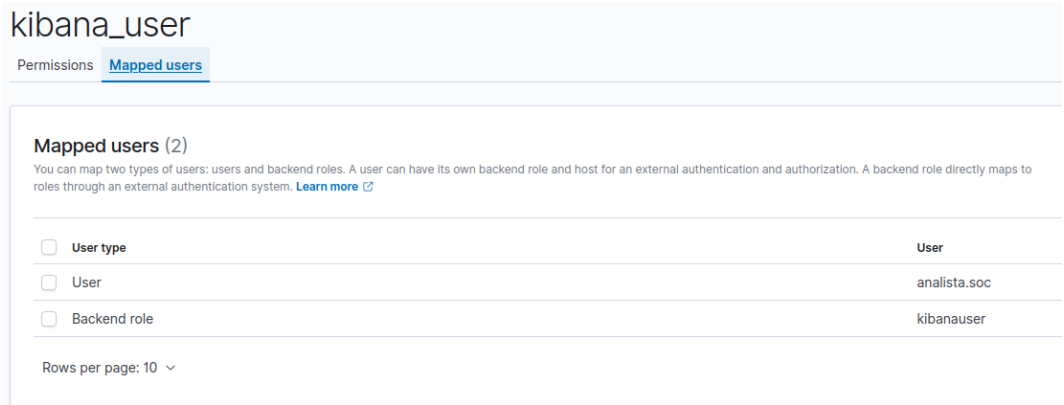


Figura 11. Rol kibana_user: analista.soc asignado.

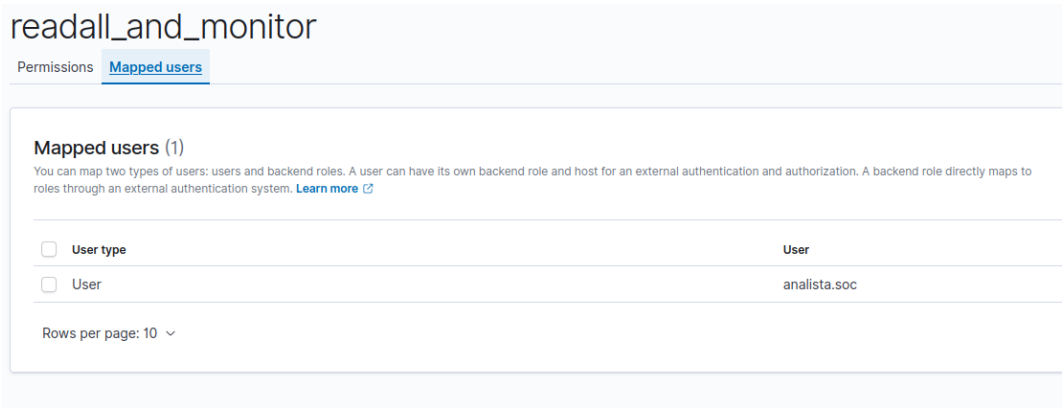


Figura 12. Rol readall_and_monitor: analista.soc asignado.

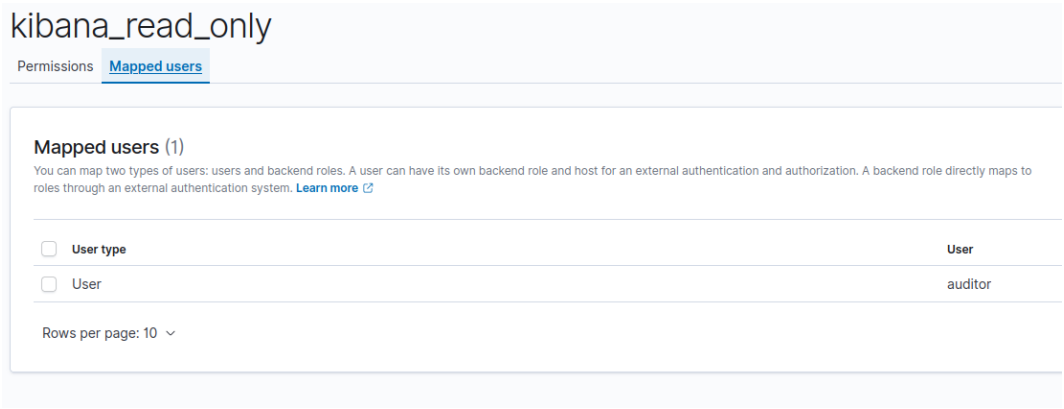


Figura 13. Rol kibana_read_only: auditor asignado.

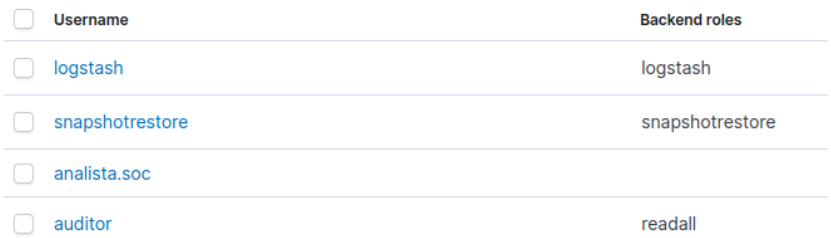


Figura 14. Backend roles: auditor lleva 'readall'. analista.soc no lleva ninguno porque sus roles se dieron por nombre.

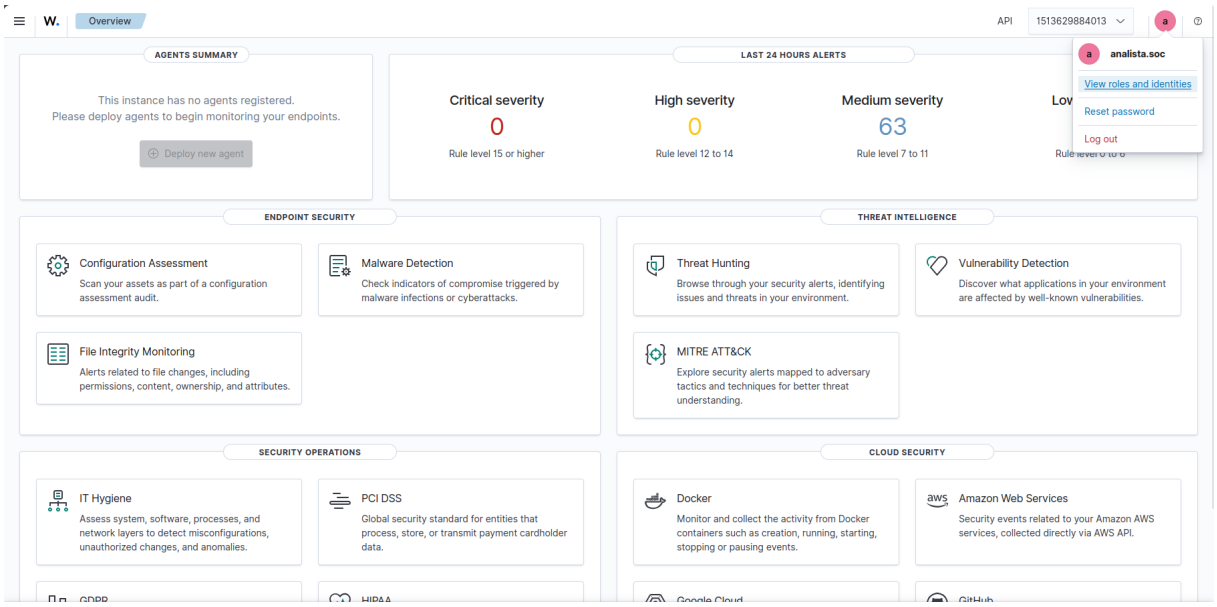


Figura 15. Inicio del dashboard entrando con analista.soc: el usuario puede abrir y usar el panel.

4. Problemas encontrados y cómo se resolvieron

Los errores se documentaron con el comando ejecutado, lo que se vio, la causa y lo que lo arregló. Donde la causa no se ha podido confirmar del todo, se indica como probable.

Problema	Qué se vio	Causa	Solución
1. docker compose no funciona	docker: unknown command: docker compose	En este equipo solo está instalado el comando con guion (docker-compose); el plugin con espacio no.	Usar docker-compose en todos los comandos.
2. Permiso denegado en Docker	permission denied while trying to connect to the docker API at unix:///var/run/docker.sock	El usuario no pertenece al grupo docker.	Poner sudo delante de los comandos de Docker.
3. Rutas y nombres distintos a la guía	La guía usa ~/wazuh-docker/single-node y single-node-wazuh.manager-1	Se trabaja con el kit del aula en ~/wazuh-soc/plataforma.	Adaptar los comandos: carpeta plataforma y contenedor plataforma-wazuh.manager-1 (nombre visto con docker ps).
4. nano no existe en el contenedor	nano: command not found	Los contenedores traen muy pocos programas.	Escribir los ficheros con cat > fichero << 'EOF' y comprobarlos con cat.
5. Dashboard: "server is not ready yet"	El navegador muestra ese mensaje. El log del dashboard repite "Request timed out" y "Unable to retrieve version information from OpenSearch nodes". El indexer respondía green desde el equipo, pero desde dentro de los contenedores (curl entre ellos) daba timeout, también desde el manager.	El tráfico entre contenedores estaba bloqueado: en iptables, FORWARD estaba en DROP y no había reglas para la red plataforma_default. Probablemente por reiniciar Docker y tocar iptables mientras se probaban cosas.	docker-compose down y docker-compose up -d, sin la opción -v. Docker recreó la red y sus reglas, el dashboard volvió a cargar y los datos (grupos, ficheros y agentes) se conservaron.
6. Forbidden al asignar el rol readall	Save error: Forbidden al mapear auditor	El mapeo de ese rol del sistema está protegido (causa exacta sin confirmar).	Se añadió el backend role readall al usuario auditor y se creó una copia del rol (readall-copy) donde sí se pudo mapear.
7. auditor no abre el panel	Application Not Found	kibana_read_only está vacío de permisos (0 de clúster, índices y tenants) y no da acceso por sí mismo. El acceso al panel lo da probablemente kibana_user, que auditor no tiene.	Decisión provisional: no darle kibana_user, porque probablemente le daría permisos de escritura sobre la configuración del panel y dejaría de ser solo lectura. Pendiente de consultar al profesor (opción: crear un rol propio de solo lectura).

Precaución aprendida: nunca usar `docker-compose down -v` ni `./bootstrap-soc.sh destroy` para "arreglar" un error: borran los volúmenes y con ellos los grupos, los agent.conf y los agentes.

Capturas de los errores

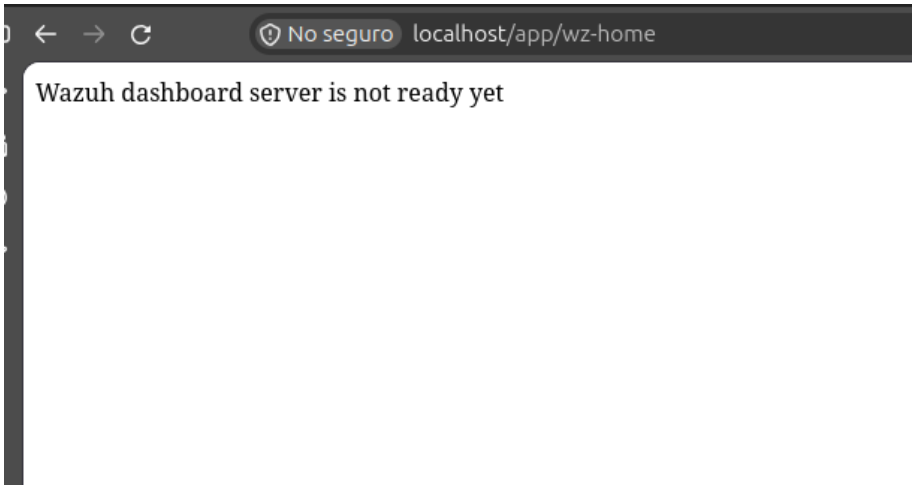


Figura 16. Problema 5: el dashboard muestra 'Wazuh dashboard server is not ready yet'.

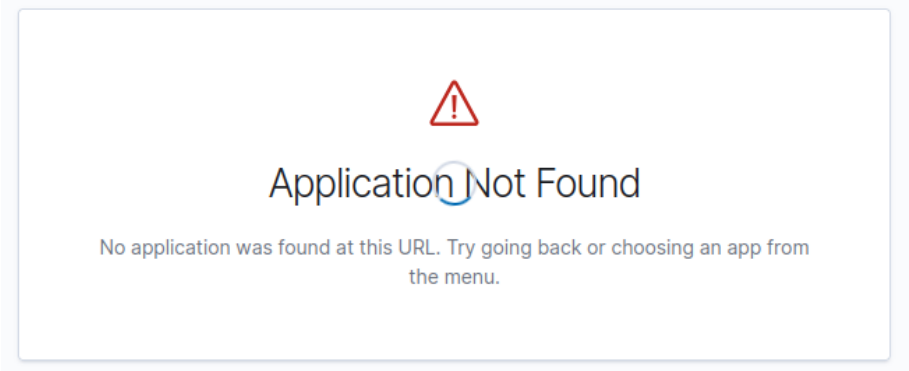


Figura 17. Problema 7: al entrar con auditor sale 'Application Not Found'.

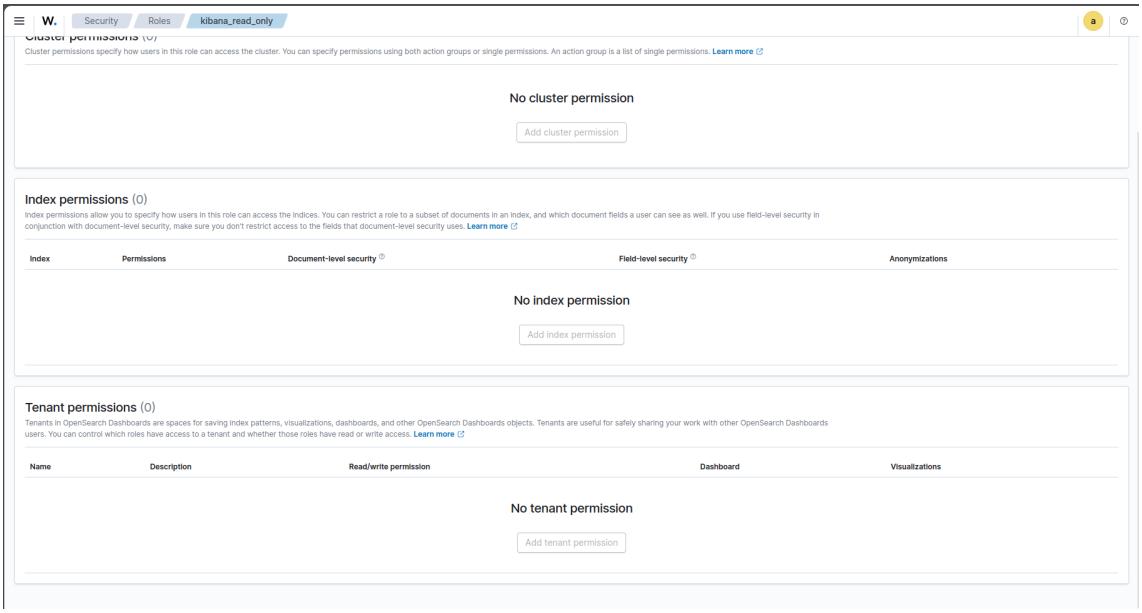


Figura 18. Problema 7: el rol kibana_read_only no tiene ningún permiso (0 en clúster, índices y tenants).

5. Estado actual frente al checklist de la práctica

Punto	Estado
Stack funcionando (indexer en green) y dashboard accesible	**Hecho**
Cuatro grupos creados	**Hecho**
agent.conf de los cuatro grupos escrito y validado (verify-agent-conf OK)	**Hecho**
Nombres de agente siguen DEPARTAMENTO-FUNCION-NN (IT-SRV-ERP, DEV-NB-01)	**Hecho**
Dos usuarios aparte de admin con roles distintos y sin permisos de administración	**Hecho** (el auditor aún no abre el panel)
Tres agentes Active de tres grupos distintos	**Pendiente** (hay dos dados de alta, sin conectar; falta el de Windows)
Labels visibles en cada agente	**Pendiente** (aparecen cuando el agente se conecta)
Ningún agente en el grupo default	**Pendiente** (queda 'paolo', un agente de prueba antiguo)
Contraseña de admin distinta a la del instalador	**Pendiente**
Contenedores al menos 30 minutos arriba sin reinicios	**Pendiente** (se recrearon durante la práctica; falta esperar)
inventario.csv con los 18 equipos	**Pendiente**

6. Próximos pasos

1. Crear los contenedores donde se conectarán los agentes y comprobar que aparecen *Active* en su grupo, con sus labels (capturas 01 y 02).
2. Generar actividad y verla en el dashboard: SSH fallido, alta de usuario, cambio en /etc y el *curl* | *bash*, que no debería detectarse (punto ciego). Después, escribir, disparar y verificar la regla local 100100.
3. Cambiar la contraseña de *admin* al final, consultando antes el método correcto del kit: si se cambia solo en el dashboard, podría cortarse el envío de alertas.
4. Redactar inventario.csv, checklist, matriz de fuentes y bitácora.
5. Decidir qué hacer con el agente 'paolo' y con el acceso del auditor al panel.

7. Qué cambiaría en un entorno real

- Rotar la contraseña de admin desde el primer día y guardar las claves en un gestor de contraseñas, nunca en ficheros de texto.
- Usar un certificado HTTPS válido en lugar del autofirmado, y no dejar el puerto del indexer (9200) abierto a toda la red.
- Crear un rol propio de solo lectura para el auditor, con acceso únicamente a los índices de alertas y al panel, en vez de reutilizar roles del sistema.
- Añadir telemetría de procesos (auditd en Linux y Sysmon en Windows) para cubrir el punto ciego de comandos como *curl* | *bash*.